Software Implementation and Testing Document

For

Group 1

Version 2.0

Authors:
Vanessa Campoe
Camryn Morris
Chloe Patrick
Wesley Jordan
Jessika Tyler
Ethan McConnell

1. Programming Languages (5 points)

JavaScript (JSX): JavaScript is the primary programming language used in our project. It is used to develop the application's functionality, including the React components, clothing selection, mannequin customization, drawing tools, and saving and loading outfits. We chose JavaScript because it works well with React and supports interactive web applications.

HTML: HTML is used to provide the basic structure of the web application through the main index.html file. We chose HTML because it is the standard markup language for web pages and works with React to display the application in a browser.

CSS: CSS is used to style the application's layout, navigation bar, side panels, buttons, and design studio. We use CSS along with Tailwind CSS to make the interface organized and user-friendly. We chose CSS because it provides flexibility in controlling the appearance of the application.

Python: Python is used in development scripts to generate and normalize mannequin SVG assets. We chose Python because it simplifies repetitive file generation and processing tasks.

2. Platforms, APIs, Databases, and other technologies used (5 points)

React: React is the primary frontend framework used to build the application's user interface. It manages components such as the design studio, clothing selection controls, and saved outfits modal.

Fabric.js: Fabric.js is used in the DesignCanvas component to create and manage the interactive drawing canvas. It supports brush tools, garment layers, clipping boundaries, and canvas serialization.

Tailwind CSS: Tailwind CSS is used throughout the frontend to style the navigation bar, side panels, buttons, and other interface elements.

Vite: Vite is used as the development server and build tool for the React application. It allows the team to run, test, and build the project efficiently.

Lucide React: Lucide React provides icons used throughout the application, including the clothing, era, and region selection controls.

Browser LocalStorage API: LocalStorage is used to save, retrieve, and delete user-created outfits directly in the browser. The saved information includes outfit names, mannequin settings, and drawing data. The project currently does not use a separate database.

JSON: JSON is used to organize the clothing catalog and store serialized outfit and canvas information.

SVG: SVG files are used for mannequin designs and clothing assets. These graphics can be displayed and scaled without losing image quality.

GitHub: GitHub is used for version control and collaboration. It allows team members to work on different features using branches, commits, and pull requests.

Node.js and npm: Node.js and npm are used in the development environment to install and manage project dependencies and run development scripts.

Vitest and jsdom: Vitest and jsdom are used for automated testing, including tests for outfit storage and canvas serialization.

Oxlint: Oxlint is used to check the JavaScript code for potential errors and maintain code quality.

3. Execution-based Functional Testing (10 points)

Execution-based functional testing was performed with Vitest v5.05 with native @vitest/coverage-v8 engine to verify the functionality of the applications storage features. Six tests were implemented, executed, and passed. These tests focused on the ability to save, load, retrieve, and delete different outfits that the user may save or have stored. The outfit needed to be correctly serialized and stored in local storage independently from one another. This way, there could be additional testing for corrupted or missing data in the storage in the garment or canvas data. The application was able to recover from the corrupted data without crashing. These tests show that the functional requirements of the program have been met for the increment.

4. Execution-based Non-Functional Testing (10 points)

Vitest v5.05 was also used for the execution-based non-functional testing of the program. Testing covered 82.97% of the statement's coverage, 78.12% of the branch's coverage, 100% of the function coverage, and 82.22% of the line coverage. The reliability of the program was tested by using corrupted JSON data, missing garment information, and invalid canvas payloads. These tests showed that the program could recover from such occurrences without crashing, showing its stability. The coverage of the testing additionally shows what portions of the program could use additional testing.

5. Non-Execution-based Testing (10 points)

Non-Execution-based Testing was handled through a review of the written code with more attention given to the code written specifically for the functions created for Increment 2 of the project. Reviewing the code made it easier to see how different portions of the program interacted with one another, which made creating the sequence diagram for the functionality of the interactions the player could have with the application much easier. The code review also made sure the program fulfilled the functional requirements by reviewing these aspects.